

UNIVERSIDADE FEDERAL DO AMAZONAS
FACULDADE DE TECNOLOGIA
ENGENHARIA ELÉTRICA - TELECOMUNICAÇÕES

Ângela Kimie Matsuo

Monitoramento de Temperatura e Umidade de Ambientes Indoor com uso de Sensores IoT e Realidade Aumentada

Manaus

2024

Ângela Kimie Matsuo

Monitoramento de Temperatura e Umidade de Ambientes Indoor com uso de Sensores IoT e Realidade Aumentada

Monografia apresentada à Coordenação do Curso de Engenharia Elétrica - Telecomunicações da Universidade Federal do Amazonas, como parte dos requisitos necessários à obtenção do título de Engenheira Eletricista com ênfase em Telecomunicações.

Orientador: Prof. Dr. Francisco de Assis Pereira Januário

Manaus

2024

Ficha Catalográfica

Ficha catalográfica elaborada automaticamente de acordo com os dados fornecidos pelo(a) autor(a).

Matsuo, Ângela Kimie
M434m Monitoramento de temperatura e umidade de ambientes indoor com uso de sensores IoT e realidade aumentada / Ângela Kimie Matsuo . 2024
57 f.: il. color; 31 cm.

Orientador: Francisco de Assis Pereira Januário
TCC de Graduação (Engenharia Elétrica - Telecomunicações) - Universidade Federal do Amazonas.

1. Sensores. 2. Realidade Aumentada. 3. Internet das Coisas. 4. Conforto Térmico. 5. Visualização. I. Januário, Francisco de Assis Pereira. II. Universidade Federal do Amazonas III. Título

Ângela Kimie Matsuo

Monitoramento de Temperatura e Umidade de Ambientes Indoor com uso de Sensores IoT e Realidade Aumentada

Monografia apresentada à Coordenação do Curso de Engenharia Elétrica da Universidade Federal do Amazonas, como parte dos requisitos necessários a obtenção de título de Engenheira Eletricista.

Aprovado em 25 de julho de 2024

BANCA EXAMINADORA

Francisco de Assis Pereira Januário
Prof. Dr. Francisco de Assis Pereira Januário
Universidade Federal do Amazonas - UFAM

Thiago Brito Bezerra
Prof. Dr. Thiago Brito Bezerra
Universidade Federal do Amazonas - UFAM

Frederico da Silva Pinagé
Prof. Dr. Frederico da Silva Pinagé
Universidade Federal do Amazonas - UFAM

Agradecimentos

Este trabalho representa a finalização de uma etapa e a realização de um sonho. Gostaria de agradecer primeiramente a Deus, além de expressar minha gratidão aos meus pais, que sempre me apoiaram.

Agradeço também aos meus familiares e amigos que acompanharam de perto e de longe a minha jornada na faculdade.

Também quero agradecer a todos os meus professores, que ao longo desses anos, compartilharam seus conhecimentos e experiências, me inspirando e guiando na jornada acadêmica. Suas orientações foram fundamentais para o meu crescimento profissional e pessoal.

“If you are the only woman in the room, use that as an opportunity to bring other women into the room ”

(Randi Zuckerberg)

Resumo

Este trabalho explora a crescente aplicação da Internet das Coisas *(IoT)* em instalações prediais e na gestão de contextos como temperatura e umidade, destacando a necessidade de apresentar os dados dos sensores *IoT* de forma acessível a supervisores prediais e outros públicos não técnicos. Propõe-se o desenvolvimento de uma camada de manipulação e visualização dos dados de sensores térmicos em ambientes indoor, utilizando realidade aumentada e integrando dados em um ambiente 3D de realidade virtual. Esta abordagem visa não apenas fornecer uma representação visual intuitiva, mas também garantir uma compreensão prática das condições de conforto térmico. Os resultados indicam que o sistema desenvolvido permite o monitoramento eficaz em tempo real das condições ambientais, proporcionando aos usuários uma ferramenta valiosa para otimizar o conforto térmico em instalações indoor, especialmente para supervisores prediais. Este estuso contribui para o uso de tecnologias inovadoras, demonstrando como a integração estratégica de *IoT*, *BIM (Building Information Modelling)* e realidade virtual pode atender às demandas práticas de gestão predial, tornando informações acessíveis e compreensíveis para diversos públicos.

Palavras-chave: Sensores, *BIM*, Conforto Térmico, Realidade Aumentada, Internet das Coisas *(IoT)*, Visualização.

Abstract

This work explores the growing application of the Internet of Things (IoT) in building installations and in the management of contexts such as temperature and humidity, highlighting the need to present IoT sensor data in a way that is accessible to building supervisors and other non-technical audiences. It is proposed to develop a layer for handling and visualising thermal sensor data in indoor environments, using augmented reality and integrating data into a 3D virtual reality environment. This approach aims not only to provide an intuitive visual representation, but also to guarantee a practical understanding of thermal comfort conditions. The results indicate that the system developed enables effective real-time monitoring of environmental conditions, providing users with a valuable tool for optimising thermal comfort in indoor facilities, especially for building supervisors. This study contributes to the use of innovative technologies, demonstrating how the strategic integration of IoT, BIM (Building Information Modelling) and virtual reality can meet the practical demands of building management, making information accessible and understandable to diverse audiences

Keywords: Sensors; BIM; Thermal comfort; Virtual Reality; Internet of Things (IoT); Visualization.

Lista de ilustrações

Figura 1 – Internet das coisas 17
Figura 2 – Realidade aumentada 18
Figura 3 – Realidade Virtual 19
Figura 4 – MQTT 20
Figura 5 – NodeMCU ESP8266 24
Figura 6 – Sensor de temperatura DHT11 25
Figura 7 – Arquitetura do sistema de monitoramento proposto 27
Figura 8 – Posicionamento dos sensores 28
Figura 9 – Transmissão de dados 29
Figura 10 – Código de leitura do sensor 32
Figura 11 – Código de definição do MQTT 32
Figura 12 – Código de publicação dos dados no MQTT 33
Figura 13 – Código de envio dos dados ao Firebase 34
Figura 14 – Plataforma unity 37
Figura 15 – Montagem do protótipo 38
Figura 16 – Recebimento de dados MQTT 39
Figura 17 – Recebimento de dados Firebase 39
Figura 18 – Unity se inscreve no tópico MQTT apropriado. 40
Figura 19 – Dashboard na realidade aumentada 41
Figura 20 – Case feita em impressora 3D. 42
Figura 21 – Protótipos 43
Figura 22 – Protótipo 44
Figura 23 – Protótipo 45
Figura 24 – Dados sendo recebidos pelo Firebase 46
Figura 25 – Gráfico dos sensores 46
Figura 26 – Dados sendo recebidos pelo Firebase 47
Figura 27 – Dashboard no celular 48
Figura 28 – Dashboard com todos os sensores enviando dados 49

Lista de tabelas

Tabela 1 – Orçamento 26

Lista de abreviaturas e siglas

BIM	*Building Information Modelling*
IoT	*Internet of things*
RA	Realidade Aumentada
RV	Realidade Virtual
MQTT	*Message Queuing Telemetry Transport*
ESP	*Espressif Systems Protocol* ou *Espressif Systems Platform*

Sumário

1 Introdução 13
1.1 Objetivos 14
1.1.1 Objetivos Específicos 14
2 Fundamentação Teórica 16
2.1 Building Information Modeling (BIM): 16
2.2 Internet das Coisas 16
2.3 Realidade Aumentada 18
2.4 Realidade virtual 19
2.5 Protocolo MQTT 20
2.6 Unity 21
2.7 Banco de dados 22
2.7.1 Principais Funcionalidades do Firebase 22
2.8 NODE MCU ESP8266 23
2.8.1 Principais Características do NodeMCU ESP8266 23
2.8.2 Vantagens do NodeMCU ESP8266 24
2.9 Sensor de temperatura DHT11 24
2.9.1 Principais Características 24
2.9.2 Funcionamento 25
3 Materiais e arquitetura 26
3.1 Proposta do sistema 27
3.1.1 Recebimento de Dados: ESP8266 28
3.1.2 Análise e Tratamento dos Dados 29
3.1.3 Transmissão dos Dados 30
3.1.4 Armazenamento dos Dados: Firebase 30
3.1.5 Visualização dos Dados: unity 31
3.2 Implementação das etapas 31
3.2.1 ESP8266 31
3.2.2 MQTT 32
3.2.3 Firebase 33
3.2.4 Unity 34
3.2.4.1 Dashboard 35
3.2.4.2 Configuração da Câmera 35
3.2.4.3 Leitura do Sensor 36
4 Montagem do protótipo 38
4.0.1 Configuração ESP e sensor DHT11 38
4.0.2 Configuração Unity 40

4.0.3 Configuração completa 41
5 Resultados 43
5.1 Protótipos montados 43
5.2 MQTT 45
5.3 Firebase 47
5.4 Dashboard unity 47
6 Conclusão 50
6.1 Trabalhos futuros 50

Referências 51

Anexos 53
ANEXO A Código para Monitoramento de Temperatura e Umidade 54

1 Introdução

Desde o surgimento da Internet das Coisas (IoT), várias arquiteturas de IoT foram implantadas para oferecer suporte ao monitoramento e gerenciamento de ambientes internos. Muitas dessa arquiteturas são construídas com o objetivo de manter as condições ideais de conforto com o auxílio de fluxos ao vivo de dados do sensor (DAVE et al., 2018). Os sensores de IoT podem fornecer dados em tempo real aos responsáveis pelo monitoramento sobre seus ambientes físicos monitorados (TEIZER et al., 2017). Esta informação será de grande importância uma vez que as condições interiores tem uma enorme influencia na qualidade do bem-estar térmico para as pessoas e equipamentos eletrônicos.

A escolha de um método apropriado para a visualização dos dados do sensor é importante, pois pode ajudar os usuários a entender intuitivamente e trabalhar com os dados de maneira mais rápida e fácil (BRESSA et al., 2022). Convencionalmente, as tecnicas de visualização estática são usadas para interpretar o significado dos dados capturados, transformando os dados em várias formas gráficas (por exemplo, gráficos de linhas, gráficos de barras, gráficos de dispersão e mapas). No entanto, essas técnicas de visualização estática têm certas restrições. Por exemplo, eles só podem permitir que determinados tipos de dados sejam apresentados e não podem apresentar os dados do sensor em tempo real, como temperatura e umidade relativa.

Assim, o problema a ser resolvido neste trabalho é: Como fornecer dados de temperatura da sala de estoque e servidores com auxílio de um dispositivo que funciona como supervisor de monitoramento por meio de um sistema visual?

O fornecimento de tais informações será de importância essencial para a tomada de decisões rápidas para enfrentar os problemas de forma mais proativa. Os métodos de visualização estática 2D tornam-se menos úteis e esses métodos precisam ser aprimorados (OLSHANNIKOVA et al., 2015). Além disso, visualizações estática sofrem significativamente com a falta de capacidade de representar dados ao vivo interativamente, proibindo assim tarefas interativas (CHANG; DZENG; WU, 2018).

A proposta deste presente trabalho é a captação de dados através de um dispositivo sensor, em seguida, a transmissão dos mesmos por redes sem fio capazes de fornecer as informações necessárias para os responsáveis através de realidade aumentada. A metodologia do projeto consiste no monitoramento de dados em tempo real de temperatura da sala dos servidores, usando a arquitetura IoT e o uso da realidade aumentada para visualização de dados para os supervisores responsáveis em monitorar o processo.

IoT e as tecnologias de sensoriamento em tempo real estão sendo cada vez mais utilizadas e aplicadas em diversas plataformas para melhorar a qualidade de vida (CHANG; DZENG; WU, 2018). Olshannikova e seus co-autores afirmou que a IoT permite que o streaming de dados de

dispositivos conectados à rede seja entregue e transformado em outros tipos de informações a serem incluídas em simulações virtuais (OLSHANNIKOVA et al., 2015).

As demais partes do artigo seguem a estrutura: No tópico 2 são demonstrados trabalhos relacionados ao tema de pesquisa que indicam métodos e ferramentas que colaboram em conhecimento para a resolução do problema tratado. No tópico 3 é mostrado claramente a arquitetura deste trabalho. No tópico 4 a arquitetura é reproduzida através da construção do protótipo com objetivo de validar os conceitos. No tópico 5 é descrito os resultados deste trabalho, expondo os dados mais relevantes. Por fim no tópico 6 é feita a conclusão e trabalhos futuros.

1.1 Objetivos

Desenvolver um sistema de monitoramento de temperatura e umidade da Sala de Estoque e dos Laboratórios de Eletrônica do Departamento de Eletrônica e Computação (DTEC), utilizando uma interface de monitoramento e visualização de dados de sensores térmicos em um ambiente interno, com uma representação virtual com tecnologias de realidade aumentada, para monitorar em tempo real as condições térmicas.

1.1.1 Objetivos Específicos

- Realizar uma pesquisa sobre as tecnologias de sensores térmicos e suas aplicações em sistemas de monitoramento ambiental. Estudar as tecnologias de realidade aumentada (RA) e como elas podem ser integradas com sistemas IoT para visualização de dados. Analisar as melhores opções para a criação de interfaces de usuário que sejam intuitivas e acessíveis para um público não técnico, especialmente supervisores da sala de estoque e servidores e outros usuários.

- Implementar o Hardware: Selecionar e adquirir sensores térmicos adequados para monitorar as condições de temperatura na Sala de Estoque e nos Laboratórios de Eletrônica do DTEC. Configurar o microcontrolador ESP8266 como concentrador para a leitura de temperatura usando sensores DHT11. Integrar os sensores térmicos com a infraestrutura de rede para que a conectividade e a transmissão de dados para o sistema seja garantida.

- Implementar o Software: Desenvolver a lógica de programação embarcada no microcontrolador ESP8266 para leitura, tratamento e transmissão dos dados dos sensores térmicos. Utilizar o Firebase para armazenamento dos dados coletados e o protocolo MQTT para a comunicação entre os dispositivos IoT. Criar uma interface de monitoramento utilizando a Unity, integrando os dados dos sensores com a representação virtual do ambiente. Programar o ambiente 3D com C#. Projetar visualizações interativas e intuitivas que tornem os dados de sensores IoT mais acessíveis e compreensíveis para os supervisores da sala de servidores e outros usuários.

- Testes: Realizar testes do sistema de monitoramento em diferentes condições ambientais para garantir a precisão e a confiabilidade dos dados coletados pelos sensores térmicos. Avaliar a funcionalidade da interface de monitoramento em realidade aumentada, verificando a usabilidade e a eficácia das visualizações interativas. Validar a integração entre os dados de sensores IoT e as tecnologias de realidade aumentada, assegurando uma experiência de monitoramento aprimorada e eficaz.

2 Fundamentação Teórica

Nesta seção serão apresentadas os conceitos e os equipamentos utilizados na elaboração do trabalho. Os conceitos a serem abordados são BIM, Internet das coisas (IoT), Realidade Aumentada (RA) e Realidade Virtual (RV), Protocolo MQTT, Banco de Dados Firebase e Desenvolvimento de Aplicações em Unity. Em relação aos equipamentos têm-se o módulo Wi-Fi NodeMCU com esp8266 e o sensor de temperatura DHT11.

2.1 Building Information Modeling (BIM):

O BIM é uma abordagem que abrange a criação e a gestão de informações detalhadas sobre um projeto de construção ao longo de todo o seu ciclo de vida. Ele é fundamental na representação virtual de edificações, permitindo o armazenamento de dados. Essa metodologia não apenas facilita a visualização e o gerenciamento dos projetos, mas também pode ser combinada com dados de sensores termicos para aprimorar o monitoramento das condições ambientais, proporcionando uma visão mais precisa e eficiente do desempenho do edifício em tempo real.

O Building Information Modeling (BIM) trouxe uma grande inovação para a indústria da construção, começando uma transformação digital que muda a forma tradicional de projetar, construir e opera edifícios. O BIM não é so sobre criar modelos 3D. Na verdade, é uma metodologia completa que junta informações detalhada e conectadas, dando uma visão mais ampla e colaborativa durante todo o ciclo de vida de um projetos de construção.

Durante a fase de concepção, o BIM capacita os profissionais da construção a criar modelos virtuais ricos em informações sobre a geometria, componentes e sistemas integrados de uma edificação. Esse abordagem multidisciplinar permite antecipar e resolver potenciais conflitos entre diferentes disciplinas, otimizando o processo de design e minimizando retrabalhos (EASTMAN et al., 2011).

2.2 Internet das Coisas

A Internet das Coisas (IoT) está transformando profundamente diversos setores ao conectar dispositivos físicos à internet para coletar e trocar dados em tempo real. Segundo (MIORANDI et al., 2012), a IoT é basicamente um sistema onde objetos do dia a dia, como sensores e atuadores, são conectado à internet para compartilhar informações e tomar decisões automáticas. Essa tecnologia é construída em três partes principais: os sensores que capturam dados do ambientes, as redes que permite a comunicação entre esses dispositivos, e os sistemas que processam esses dados para gerar percepção úteis.

Os sensores IoT podem ser encontrados em praticamente tudo hoje em dia, desde aparelhos domésticos inteligentes até equipamentos industriais complexos. Eles coletam uma variedade de informações, como temperatura, umidade, movimento, e até mesmo dados de localização. As redes de comunicação, por sua vez, são responsáveis por garantir que esses dados sejam transmitidos de forma segura e rapida para onde precisam ir. Isso pode ser feito atraves de redes sem fio, como Wi-Fi ou Bluetooth, ou até mesmo atraves de redes celulares ou satélites em casos mais específicos.

Uma vez que os dados são coletados, eles são processados em sistemas computacionais, geralmente na nuvem ou em dispositivos de computação próxima (*edge computing*). É nesses sistemas que algoritmos inteligentes analisam o dado para detectar padrões, prever comportamentos futuros, ou até mesmo controlar dispositivos automaticamente com base nas informações recebidas. Esse processo não so melhora a eficiência dos processos industriais e urbanos, mas também pode transformar como interagimos com nossas casas e cidades.

A IoT não é apenas sobre conectar dispositivos; ela está impulsionando novos modelos de negócios e melhorando nossa qualidade de vida. Ela permite desde automatizações simples, como ajustar a temperatura em uma casa automaticamente, até soluções complexa, como criar cidades inteligentes que consomem menos recursos e oferecem serviços mais eficientes para os cidadãos.

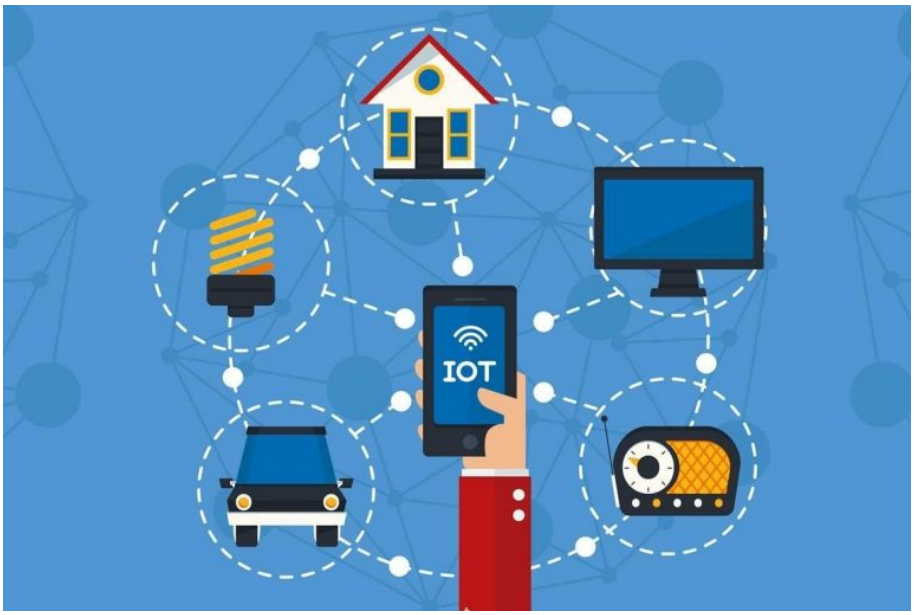


Figura 1 – Internet das coisas

Fonte: napratica.org.br.

2.3 Realidade Aumentada

A Realidade Aumentada (RA) é uma tecnologia que adiciona informações digitais, como imagens, vídeos e sons, ao que estamos vendo no mundo real. Diferente da realidade virtual (RV), que cria um ambiente totalmente digital, a RA comlementa o ambiente fisico com dados extras que podemos visualizar através de dispositivos como celulares, tablets ou óculos.

Na manutenção industrial, a RA tem várias aplicações úteis. Por exemplo, ela pode fornecer instruções detalhadas e visuais para técnicos de manutenção. Assim, um técnico pode ver exatamente onde cada peça deve ser colocada ou como consertar um equipamento, com sobreposições digitais que aparecem sobre o que está sendo visto.

Um exemplo é o Sistema de Orientação por RA, que ajuda funcionários novos ou inexperientes a realizar tarefas complexas com instruções passo a passo. Isso digitaliza e melhora as instruções tradicionais em papel, adicionando imagens e videos para facilitar o entendimento. Com isso, é possível reduzir o tempo necessário para completar as tarefas (KIM et al., 2020).

Alem disso, a RA pode ser usada para monitorar condições em tempo real. Por exemplo, sensores de temperatura podem mostrar a temperatura atual dos equipamentos e ambiente diretamente na tela, sem precisar se deslocar até o local para fazer a medição manualmente. Isso ajuda a identificar problemas mais rapido e evita deslocamento desnecessarios.

Em resumo, a Realidade Aumentada é uma tecnologia incrível que pode transformar muitos setores, incluindo a manutenção industrial, oferecendo informações adicionais e suporte visual no próprio local de trabalho, como pode ser visto na figura 5.

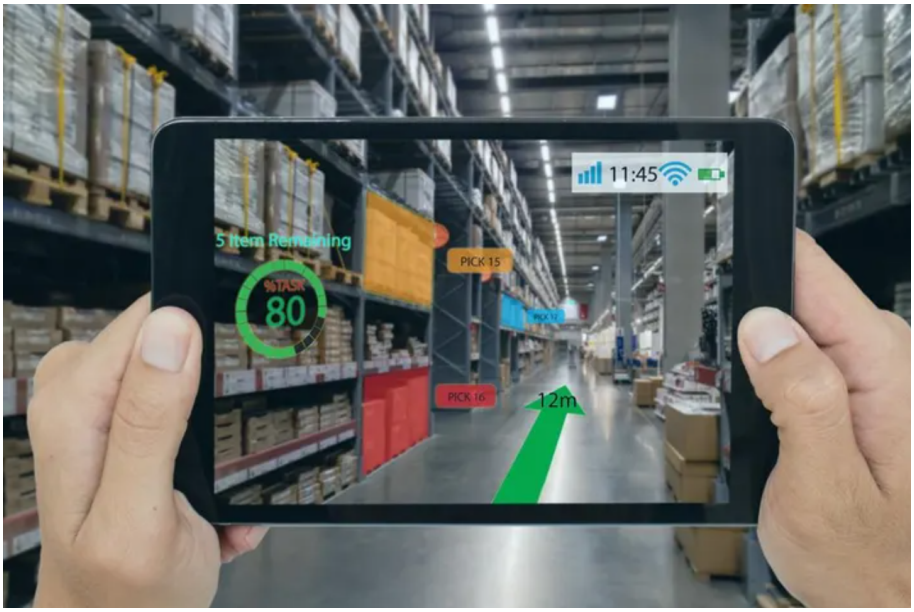


Figura 2 – Realidade aumentada

Fonte: forbes.com.

2.4 Realidade virtual

A Realidade Virtual (RV) é uma tecnologia que cria um ambiente digital totalmente imersivo, permitindo que os usuários interajam com um espaço 3D gerado por computador. Diferente da realidade aumentada, que sobrepõe informações digitais ao mundo real, a RV transporta o usuário para um ambiente completamente novo, simulando uma experiência que pode parecer muito real.

O conceito de RV começou a se desenvolver na segunda metade do século XX, com avanços significativos na tecnologia desde então. Na década de 1990, inovações como a criação de modelos 3D e ambientes virtuais mais complexos marcaram um ponto de virada. Atualmente, a RV é amplamente utilizada em diversas áreas, desde jogo e entretenimento até aplicações industriais e de engenharia.



Figura 3 – Realidade Virtual

Fonte: HIHUB.TECH.

Um exemplo interessante de como a RV pode ser aplicada é no monitoramento de sensores de temperatura. Em um ambiente industrial onde sensores de temperatura estão instalados em diferentes máquinas e equipamentos. A RV pode ser usada para criar uma simulação virtual do ambiente, permitindo que engenheiros e técnicos visualizem as leituras de temperatura em tempo real dentro de um modelo 3D.

De acordo com (WU, 2023), "a tecnologia de realidade virtual tem vantagens únicas, como a capacidade de criar experiências imersivas que não requerem muita participação física e têm aplicações positivas em áreas como educação, entretenimento e saúde."Isso mostra como a RV tem se tornado uma ferramenta valiosa para diversas aplicações, incluindo a visualização e análise de dados de sensores.

A Realidade Virtual é uma tecnologia que está transformando a forma como interagimos com dados, como os de sensores de temperatura, proporcionando uma maneira imersiva de explorar sistemas e processos no mundo digital.

2.5 Protocolo MQTT

MQTT (Message Queuing Telemetry Transport) é um protocolo de comunicação amplamente utilizado em redes de sensores e sistemas distribuídos. Ele foi desenvolvido para ser leve e eficiente, o que o torna perfeito para ambientes com pouca largura de banda e dispositivos com recursos limitados.

O MQTT usa um modelo de comunicação chamado publish/subscribe. Isso significa que as mensagens são enviadas para tópicos específicos e os dispositivos que estão interessados se inscrevem nesses tópicos para receber as mensagens. Esse método é muito útil em redes que são dinâmicas e distribuídas, como as redes de sensores sem fio (WSNs).

No artigo de Hunkeler, o protocolo MQTT é descrito como uma solução eficaz para lidar com a complexidade das redes de sensores, onde "a comunicação pode ser desafiadora devido à dinâmica das redes, como mudanças de endereço e falhas de dispositivos" (HUNKELER; TRUONG; STANFORD-CLARK, 2008). O modelo publish/subscribe do MQTT permite que a comunicação não dependa de endereços de rede específicos, mas sim do conteúdo das mensagens e dos interesses dos dispositivos.

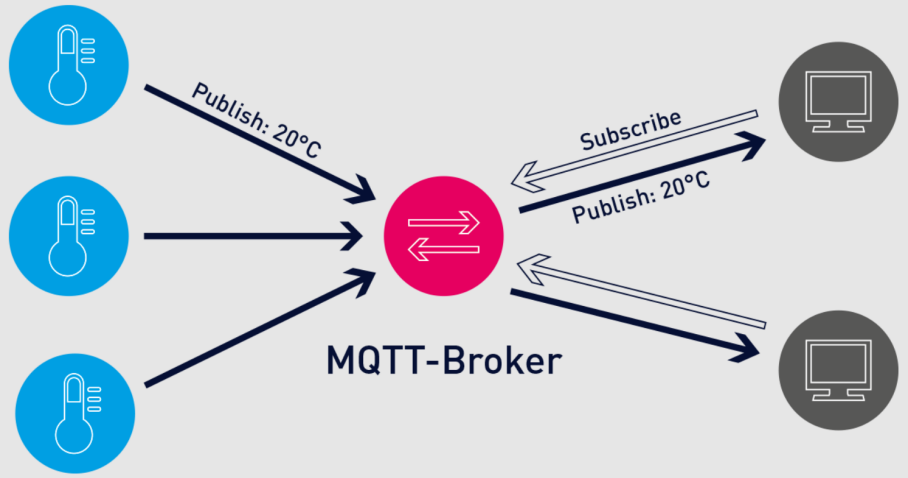


Figura 4 – MQTT

Fonte: paessler.com.

Além disso, o MQTT facilita a integração das redes de sensores com redes tradicionais, como a Internet e LANs, permitindo que os dados coletados pelos sensores sejam enviados para aplicações em redes maiores. Isso é muito importante para usar os dados de forma prática e

para desenvolver soluções inovadoras em áreas como automação industrial e monitoramento ambiental.

Em resumo, o MQTT é uma solução importante para a comunicação em redes com recursos limitados e baixa largura de banda. Ele oferece um modelo de comunicação eficiente e flexível, perfeito para atender às necessidades das redes de sensores modernas.

2.6 Unity

A Realidade Aumentada (RA) e Realidade Virtual(RV) está se tornando uma tecnologia muito importante, permitindo novas maneiras de interagir com o mundo digital e físico. O Unity é uma das principais plataformas para desenvolvimento de jogos e simulações, é uma ferramenta para criar aplicações desse tipo. Ele é muito útil para criar experiências imersivas porque suporta várias funcionalidades de AR através de integração com ferramentas e SDKs como AR Foundation, Vuforia e ARKit/ARCore. Isso faz com que os desenvolvedores façam aplicativos de AR que funcionem em diferentes dispositivos e sistemas operacionais, aproveitando as capacidades específicas de cada um.

Uma das maiores vantagens do Unity é sua interface gráfica, que oferece um ambiente de desenvolvimento visual completo e fácil de usar. Isso reduz a necessidade de programar coisas complicadas e acelera o processo de desenvolvimento, fazendo com que seja mais acessível para todos os desenvolvedores. Além disso o Unity permite o desenvolvimento multiplataforma, o que significa que as aplicações de RA podem ser distribuídas em vários sistemas operacionais, como iOS e Android. Isso amplia o alcance dos aplicativos e garante que eles possam atingir uma base geral de usuários.

Outra grande qualidade do Unity é sua capacidade de renderização avançada, que inclui efeitos visuais impressionantes, iluminação realista e simulação física precisa. Esses recursos são essenciais para criar experiências de RA visualmente atraentes e envolventes, que podem impactar significativamente a forma como os usuários interagem com o ambiente digital e físico. A plataforma também tem uma comunidade ativa e uma vasta biblioteca de recursos, como tutoriais e fóruns, que oferecem suporte e ajudam a resolver problemas durante o desenvolvimento.

O Unity é uma plataforma robusta e flexível para o desenvolvimento de aplicações de Realidade Aumentada. Sua integração com vários SDKs de AR, juntamente com uma interface intuitiva e suporte multiplataforma, faz dele uma ferramenta ideal para criar experiências de RA imersivas. A combinação de recursos avançados e uma comunidade ativa contribui para que o Unity proporcione soluções inovadoras e impactantes no campo da RA. Como mencionado em um estudo, "A realidade aumentada (RA) é um tipo de extensão virtual (aparente) da realidade, quando, por exemplo, elementos virtuais podem ser projetados no ambiente real ao olhar ao redor com a câmera de um telefone móvel ou usando óculos criados para esse fim"(SIMON, 2023), demonstrando a flexibilidade e a aplicabilidade do Unity em diferentes dispositivos e

cenários de uso.

2.7 Banco de dados

O Firebase é uma plataforma desenvolvida pelo Google que fornece serviços backend para aplicativos móveis e web. Lançado originalmente em 2011, o Firebase tem se destacado por oferecer soluções robustas e escaláveis para o desenvolvimento de aplicativos, com um foco especial em real-time data, autenticação e análises.

2.7.1 Principais Funcionalidades do Firebase

- Firebase Realtime Database e Cloud Firestore:
  - Firebase Realtime Database: É um banco de dados NoSQL que armazena e sincroniza dados em tempo real. As atualizações são instantaneamente refletidas em todos os clientes conectados, o que é ideal para aplicativos que necessitam de atualizações contínuas e em tempo real, como aplicativos de mensagens e de monitoramento.
  - Cloud Firestore: É um banco de dados NoSQL mais moderno e flexível que complementa o Realtime Database. Oferece uma estrutura de dados mais rica e suporte a consultas complexas, além de melhor integração com outras ferramentas do Firebase e Google Cloud.
- Cloud Functions: Permitem a execução de código backend em resposta a eventos disparados por Firebase ou outros serviços de nuvem. Isso é útil para implementar lógica de negócios personalizada, como validações de dados, processamento de pagamentos ou envio de notificações.
- Firebase Hosting: Fornece uma plataforma rápida e segura para hospedar sites e aplicativos web. Oferece recursos como entrega de conteúdo via CDN (Content Delivery Network), SSL gratuito e implantação continua através de uma interface de linha de comando.

O Firebase é amplamente utilizado para desenvolver aplicativos para celular e web devido as suas características de tempo real e integração simplificada. As principais vantagens incluem:

- Desenvolvimento Rápido: Com a ampla gama de ferramentas e serviços oferecidos, os desenvolvedores podem reduzir significativamente o tempo necessário para criar, testar e lançar aplicativos.
- Escalabilidade: O Firebase é projetado para suportar desde pequenos aplicativos até grandes aplicativos com milhões de usuários, garantindo que as soluções sejam escaláveis conforme a necessidade.

- Facilidade de Integração: A plataforma oferece integração simplificada com outras ferramentas do Google Cloud e com muitos frameworks de desenvolvimento, o que facilita a criação de soluções complexas e integradas.

O Firebase é uma solução abrangente para o desenvolvimento de aplicativos, oferecendo serviços backend que facilitam a construção de aplicativos modernos e interativos. Com suas funcionalidades de banco de dados em tempo real, autenticação, análise e muito mais, o Firebase é uma escolha popular para desenvolvedores que buscam uma plataforma completa para suas necessidades de desenvolvimento.

2.8 NODE MCU ESP8266

O NodeMCU ESP8266 é uma placa de desenvolvimento popular para projetos de Internet das Coisas (IoT). Desenvolvido pela Espressif Systems, o ESP8266 é um microcontrolador que integra conectividade Wi-Fi, oferecendo uma solução compacta e econômica para projetos que requerem comunicação de rede sem fio.

2.8.1 Principais Características do NodeMCU ESP8266

O ESP8266 é equipado com um processador de 32 bits com arquitetura RISC, operando a uma frequência de até 80 MHz (ou 160 MHz em modelos mais recentes), sendo adequado para a maioria dos projetos de IoT e automação. Sua conectividade Wi-Fi embutida permite que a placa se conecte a redes sem fio e se comunique com servidores e outros dispositivos pela internet, eliminando a necessidade de módulos Wi-Fi externos e simplificando o design dos projetos.

A placa NodeMCU é compatível com o Ambiente de Desenvolvimento Integrado (IDE) Arduino e com o firmware NodeMCU baseado em Lua, facilitando a programação e o desenvolvimento de projetos para aqueles familiarizados com a plataforma Arduino. Além disso, o NodeMCU ESP8266 oferece vários pinos de entrada e saída digitais, um pino analógico e interfaces de comunicação como UART, SPI e I2C, permitindo a conexão com sensores, atuadores, displays e outros periféricos.

Com memória Flash variando entre 4 MB e 16 MB, dependendo do modelo, e RAM suficiente para executar programas de forma eficiente, o ESP8266 armazena o firmware e os dados do programa na memória Flash. O microcontrolador é amplamente suportado por diversas plataformas de nuvem e serviços de IoT, facilitando a integração com sistemas de monitoramento e controle remoto, permitindo que os dados sejam enviados para a nuvem para análise, armazenamento e visualização.

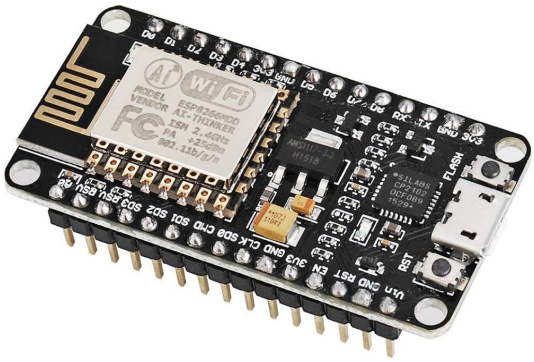


Figura 5 – NodeMCU ESP8266

Fonte: amazon.com.

2.8.2 Vantagens do NodeMCU ESP8266

Uma das principais vantagens do ESP8266 é sua relação custo-benefício. Ele oferece conectividade Wi-Fi a um preço acessível, se tornando uma solução econômica para projetos de IoT. A facilidade de programação, devido à compatibilidade com o IDE Arduino e o firmware Lua, simplifica o desenvolvimento, tornando o NodeMCU acessível tanto para iniciantes quanto para profissionais. Além disso, sua compactabilidade e versatilidade, com uma ampla gama de pinos e interfaces, o tornam adequado para diversos tipos de projetos e aplicações.

2.9 Sensor de temperatura DHT11

O DHT11 é um sensor amplamente utilizado para medir a temperatura e a umidade do ambiente, conhecido por sua simplicidade e custo acessível. Ele combina dois tipos de sensores em um único módulo: um sensor capacitivo para medir a umidade relativa e um termistor para medir a temperatura.

2.9.1 Principais Características

O DHT11 possui uma faixa de medição de temperatura que varia de 0°C a 50°C com uma precisão de ±2°C, e uma faixa de medição de umidade de 20% a 80% com uma precisão de ±5% RH. Esses parâmetros o tornam adequado para aplicações onde a precisão extrema não é necessária, mas onde a facilidade de uso e o custo reduzido são importantes.

O sensor utiliza uma interface de comunicação digital de 1 fio, o que simplifica a conexão e a leitura dos dados pelo microcontrolador. Esse recurso permite que o DHT11 transmita dados digitais de temperatura e umidade através de um único pino, facilitando a integração com diversos sistemas.

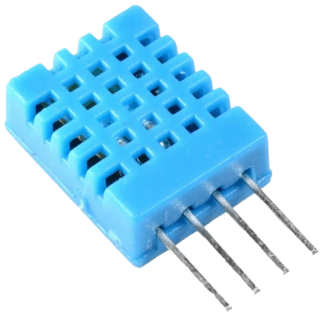


Figura 6 – Sensor de temperatura DHT11

Fonte: makerhero.com.

2.9.2 Funcionamento

O funcionamento do DHT11 se baseia em um sensor capacitivo para medir a umidade e um termistor para medir a temperatura. O sensor capacitivo detecta mudanças na capacitância devido às variações na umidade relativa do ar, enquanto o termistor mede as variações na resistência em função da temperatura.

O processo de medição inicia com o sensor enviando um sinal de início seguido por pacotes de dados digitais, que são decodificados pelo microcontrolador. Cada pacote de dados inclui 40 bits, correspondentes a dois bytes de umidade e dois bytes de temperatura. O microcontrolador interpreta esses dados e os converte em valores utilizáveis, realizando verificações e cálculos adicionais para garantir a precisão.

Apesar de suas vantagens, o DHT11 apresenta algumas limitações. A precisão e a faixa de medição são mais restritas quando comparadas a sensores mais avançados, como o DHT22.

O DHT11 é um sensor básico e econômico que proporciona uma solução prática para medir temperatura e umidade. Sua integração simples e custo reduzido o tornam uma escolha adequada para uma variedade de aplicações básicas e projetos educacionais.

3 Materiais e arquitetura

Para o desenvolvimento do projeto, foi escolhido o microcontrolador ESP8266 como concentrador principal. Além disso, foi utilisado quatro modulos do sensor DHT11 para a leitura da temperatura. A escolha do ESP8266 foi estrategica devido a sua capacidade de enviar dados via wi-fi, além de ser compativel com codigos escritos em linguagens de programação C, oferecendo flexibilidade e eficiência no desenvolvimento, que são importants para o controle dos sensores.

Os componentes utilizados nesse projeto foram escolhidos por sua robustez e facilidade de utilização em relação a outros componentes mais baratos, no entando podem ser modificados para similares. A tabela 1 mostra o orçamento para os materiais desse trabalho, onde todos os valores foram pesquisados no mercado livre.

Tabela 1 – Orçamento

Componente	**Preço**
ESP	R$ 32.50 x 4
DHT11	R$ 13.20 x 4
Carregador	R$ 33.89 x 4
Cabo Micro USB	R$ 20.61 x 4
Total	R$ 400.08

O ambiente virtual necessário para a visualização e interação com os dados foi criado utilizando a ferramenta Unity. Unity foi selecionada por oferecer um conjunto geral de bibliotecas e ferramentas que facilitam a comunicação com sistemas embarcados em IoT. Sua capacidade de criar ambientes 3D detalhados e interativos foi importante para o sucesso do projeto. Na programação do ambiente 3D, utilizamos a linguagem C#, que é a linguagem principal do Unity. O projeto de (DAS, 2021) foi utilizado como base para a criação do ambiente virtual.

Para a comunicação entre o ambiente virtual e o concentrador ESP8266. O protocolo MQTT (Message Queuing Telemetry Transport) foi escolhido por sua eficiência e baixo consumo de largura de banda, o que é ideal para aplicações em IoT. A biblioteca uPLibrary.Networking.M2Mqtt facilita a implementação desse protocolo, permitindo a troca de mensagens entre o Unity e os dispositivos ESP.

A arquitetura do projeto foi planejada com base na literatura para que todos os componentes e sistemas funcionassem uniformemente, resultando em um sistema eficiente e eficaz para a leitura e visualização de dados de temperatura em tempo real.

3.1 Proposta do sistema

A arquitetura do projeto foi cuidadosamente desenvolvida para integrar a camada física do IoT com RA, visando proporcionar uma visualização precisa e intuitiva dos dados de sensores térmicos. Utilizando o microcontrolador ESP8266 como concentrador e para a leitura de temperatura com sensores DHT11, foi criado um sistema que recebe, analisa e transmite os dados coletados. Esses dados são então visualizados em um ambiente virtual criado com Unity.

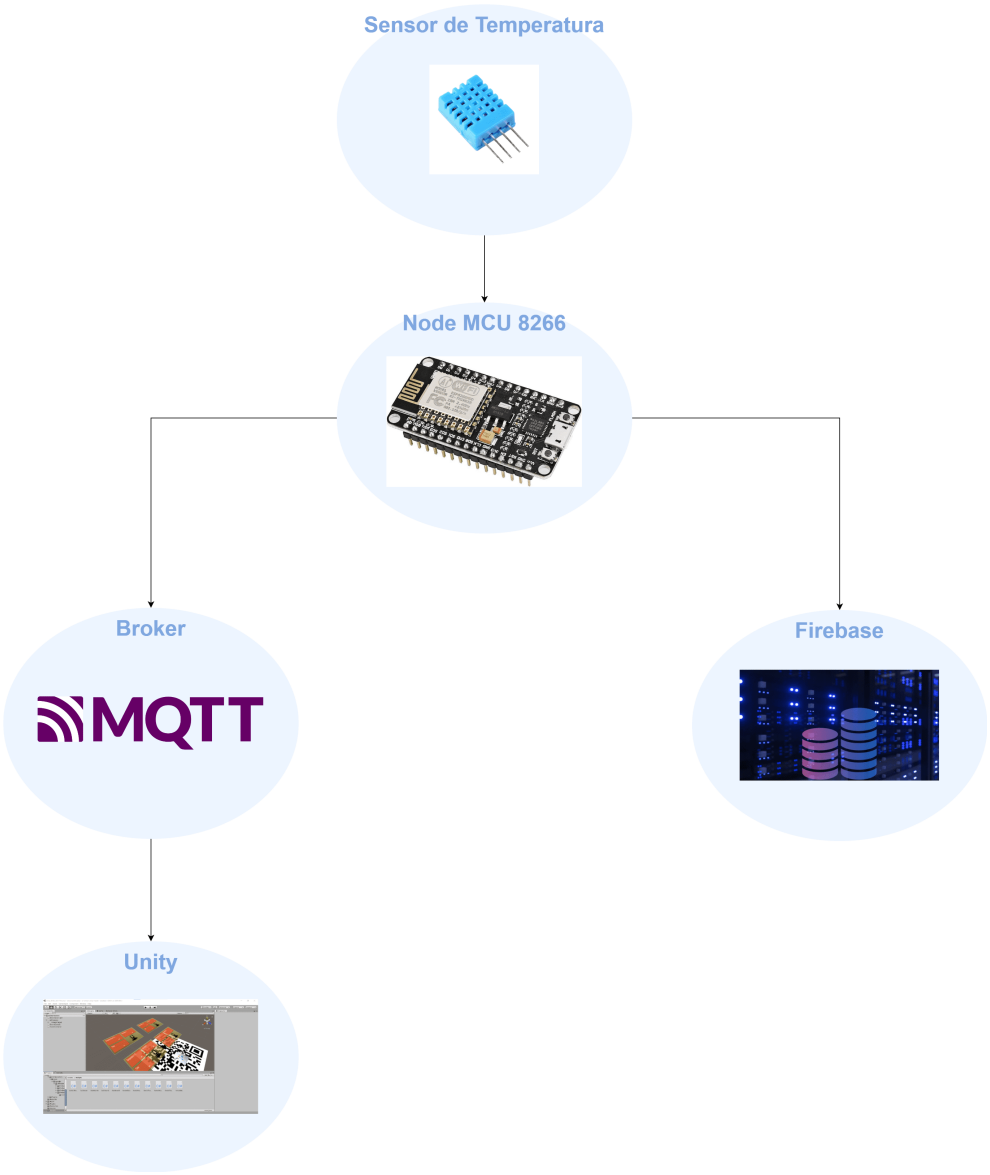


Figura 7 – Arquitetura do sistema de monitoramento proposto

O projeto de monitoramento é composto por quatro sensores de temperatura e umidade posicionados estrategicamente na sala dos servidores para garantir um controle eficaz das condições ambientais como pode ser visto na figura 8. O sistema é desenvolvido para fornecer uma solução completa que abrange desde a coleta dos dados dos sensores até sua visualização

em um ambiente virtual.

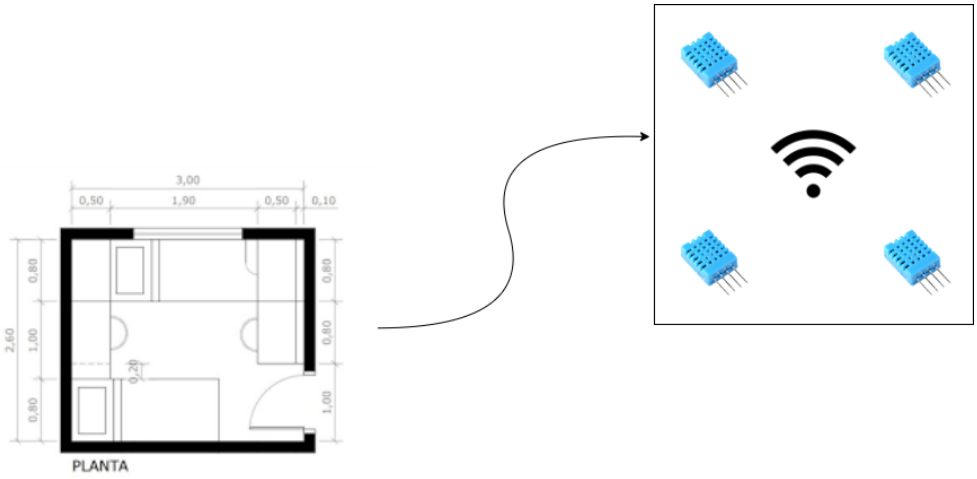


Figura 8 – Posicionamento dos sensores

A arquitetura do sistema é dividida em cinco etapas principais: recebimento dos dados do sensor, análise e tratamento dos dados, transmissão dos dados, armazenamento dos dados e visualização dos dados no ambiente virtual. Cada uma dessas etapas é essencial para garantir que a informação apresentada seja precisa, eficiente e útil.

3.1.1 Recebimento de Dados: ESP8266

A primeira etapa do sistema envolve a instalação de sensores IoT, especificamente sensores de temperatura, para coletar dados em tempo real sobre variáveis ambientais. O processo de recepção de dados é formalizado em um fluxograma que detalha a transmissão dos dados de temperatura. Quando o sensor é ativado, verifica-se o intervalo de recepção dos dados. Se este intervalo for de um minuto, o algoritmo armazena os dados coletados para posterior análise.

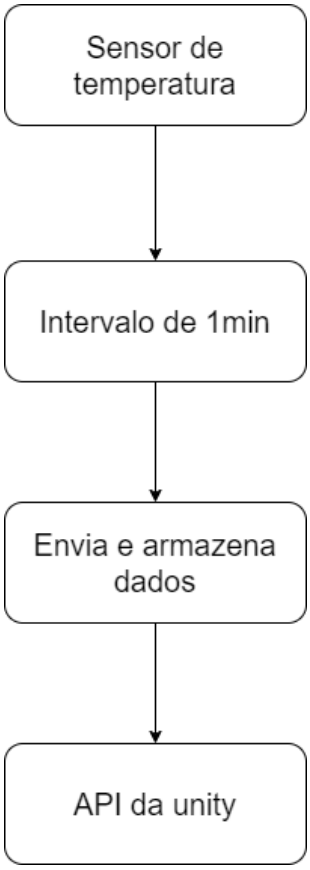


Figura 9 – Transmissão de dados

Os sensores DHT11, que medem a temperatura e a umidade do ambiente, são conectados a microcontroladores ESP8266. Cada sensor é posicionado em um local específico da sala dos servidores para garantir uma cobertura ampla e representativa das condições ambientais. Os microcontroladores são responsáveis pela leitura dos dados dos sensores em intervalos regulares. A comunicação entre o sensor e o microcontrolador é realizada através de pinos digitais, onde o microcontrolador executa uma série de comandos para solicitar e receber as medições de temperatura e umidade.

Os dados coletados pelos sensores são enviados para o microcontrolador, que executa a leitura e o tratamento inicial dos dados. É crucial garantir que as leituras sejam precisas e que não haja falhas na comunicação. Caso o sensor não responda adequadamente, o sistema pode tentar realizar a leitura novamente ou registrar uma mensagem de erro.

3.1.2 Análise e Tratamento dos Dados

Na segunda etapa, os dados recebidos dos sensores de temperatura são agregados e analisados para proporcionar uma visão contextual e indicativa do status do ambiente monitorado. Esta etapa envolve:

- Coleta e seleção dos dados dos sensores;
- Separação de cada sensor: sensor 1, sensor 2, sensor 3 e sensor 4.

Depois que os dados são recebidos do sensor, o microcontrolador realiza uma análise inicial. Isso inclui a verificação da integridade dos dados lidos. Os valores de temperatura e umidade são checados para garantir que não estejam fora do intervalo esperado ou apresentem valores inválidos, como NaN (*Not a Number*).

Se os dados estiverem válidos, o microcontrolador os formata em um formato apropriado para transmissão. Por exemplo, os valores podem ser convertidos em strings ou números com precisão adequada.

No contexto deste projeto, a análise é focada em uma sala de servidores local, onde o monitoramento da temperatura é importante para a manutenção adequada do ambiente.

3.1.3 Transmissão dos Dados

A terceira etapa envolve a escolha e implementação de protocolos de comunicação para a transmissão dos dados. O protocolo MQTT (Message Queuing Telemetry Transport) foi selecionado por sua eficiência em gerenciar pacotes de dados leves e pela capacidade de operar em uma arquitetura Publish/Subscribe, onde os dispositivos conectados podem atuar como publicadores ou assinantes. No projeto, um broker gerencia as informações recebidas dos sensores e as distribui para os usuários inscritos, que são os assinantes.

Os dados processados são então preparados para transmissão para um sistema centralizado. Este processo utiliza dois métodos principais:

- MQTT (Message Queuing Telemetry Transport): O protocolo MQTT é utilizado para a transmissão eficiente dos dados em tempo real. Cada microcontrolador publica os dados em topicos específicos de um broker MQTT. Estes tópicos são definidos para diferentes tipos de dados, como temperatura e umidade. O MQTT é um protocolo leve e ideal para a transmissão de dados em ambientes com largura de banda limitada, como redes de sensores. O microcontrolador se conecta ao broker MQTT, e os dados são enviados para topicos específicos, onde outros dispositivos ou aplicações podem se inscrever para receber as atualizações.
- Firebase: Além do MQTT, os dados são enviados para o Firebase Realtime Database, que vai ser discutido no próximo tópico.

3.1.4 Armazenamento dos Dados: Firebase

A quarta etapa do projeto é o armazenamento dos dados em um banco de dados. Para isso, foi escolhido o Firebase, que é uma plataforma de desenvolvimento de aplicativos para

celular e web que oferece um banco de dados NoSQL em tempo real. Os dados de temperatura coletados são enviados para o Firebase, onde são armazenados e podem ser acessados a qualquer momento para análise posterior. O uso do Firebase garante que os dados estejam seguros e disponíveis em tempo real para qualquer aplicação que precise acessá-los.

O Firebase permite que os dados sejam sincronizados instantaneamente entre o microcontrolador e a plataforma de armazenamento. Os dados, incluindo temperatura, umidade e hora da leitura, são enviados para o banco de dados e armazenados em uma estrutura de árvore que facilita o acesso e a organização. O Firebase também oferece funcionalidades de autenticação e segurança para garantir que os dados sejam protegidos.

3.1.5 Visualização dos Dados: unity

A Unity é uma plataforma de desenvolvimento de jogos e aplicações em 3D, amplamente utilizada em diversas indústrias além do entretenimento. Originalmente projetada para criar jogos, a Unity evoluiu para uma ferramenta multifuncional que atende a desenvolvedores de realidade aumentada (RA), realidade virtual (RV), simulações, visualizações arquitetônicas, e muito mais.

A etapa final do projeto é a visualização dos dados no ambiente virtual 3D. A aplicação de RA permite a visualização dos dados em tempo real, oferecendo uma interface interativa e intuitiva. Os painéis virtuais mostram os valores recebidos dos sensores, cada painel mostra um sensor, bem como a hora atual.

Integração de Dados: Os dados armazenados no Firebase e transmitidos via MQTT são integrados ao ambiente virtual. A integração é feita através de APIs ou interfaces de programação que permitem a recuperação dos dados e sua exibição no modelo 3D. Esta visualização ajuda a identificar rapidamente áreas com condições ambientais irregulares e facilita a tomada de decisões para ajustar o clima da sala dos servidores conforme necessário.

3.2 Implementação das etapas

3.2.1 ESP8266

A parte do código que lida com o recebimento de dados do sensor DHT11 no ESP8266 é importante pois, ler a temperatura e a umidade do ambiente e, posteriormente, enviar esses dados para o Firebase e para o broker MQTT. A inicialização do sensor DHT11 é feita dentro da função setup(), onde o sensor é configurado para começar a operar com esse comando: "dht.begin();"

A leitura dos dados do sensor DHT11 é realizada dentro da função loop(), que é chamada para manter o programa rodando continuamente. Antes de ler os dados do sensor, o código garante que o ESP8266 esteja conectado ao broker MQTT. Após garantir a conexão MQTT, o código lê os dados de temperatura e umidade do sensor DHT11.

```
float temperatura = dht.readTemperature();
float umidade = dht.readHumidity();

if (isnan(temperatura) || isnan(umidade)) {
  Serial.println("Falha ao ler do sensor DHT!");
  return;
}
```

Figura 10 – Código de leitura do sensor

- dht.readTemperature(): Lê a temperatura ambiente do sensor DHT11.
- dht.readHumidity(): Lê a umidade do sensor DHT11.
- isnan(temperatura) || isnan(umidade): Verifica se as leituras são válidas. Se alguma das leituras for NaN (*Not a Number*), indica uma falha na leitura, e o código imprime uma mensagem de erro e retorna para tentar novamente no proximo ciclo.

3.2.2 MQTT

A parte do código que lida com o MQTT é responsável por configurar a comunicação entre o ESP8266 e um broker MQTT. Essa comunicação permite que o ESP8266 publique mensagens com dados de sensores em tópicos específicos, que podem ser subscritos por outros dispositivos ou serviços.

```
const char* mqtt_server = "broker.hivemq.com";
const char* mqtt_topic = "esp8266/sensor1";
const char* mqtt_topic_temp = "esp8266/sensor1/temperatura";
const char* mqtt_topic_hum = "esp8266/sensor1/umidade";

WiFiClient espClient2;
PubSubClient client(espClient2);
```

Figura 11 – Código de definição do MQTT

- mqtt_server: Define o endereço do broker MQTT.
- mqtt_topic: Define o tópico principal onde as mensagens serão publicadas.
- mqtt_topic_temp e mqtt_topic_hum: Define os tópicos específicos para publicar as leituras de temperatura e umidade, respectivamente.

- WiFiClient espClient2: Cria um cliente WiFi para conectar ao broker MQTT.
- PubSubClient client(espClient2): Inicializa o cliente MQTT utilizando o cliente WiFi.

No algoritmo do programa, foram necessárias algumas bibliotecas para o funcionamento do protocolo MQTT, como a ESP8266WiFi.h, FirebaseESP8266.h e PubSubClient.h. Essas bibliotecas permitem a transmissão dos dados via Wi-Fi e a publicação de informações no tópico do servidor e no banco de dados. Os tópicos do broker utilizados no projeto são "esp8266/sensor1/temperatura"e "esp8266/sensor1/umidade"para o primeiro sensor, que recebem os dados com os valores de temperatura e umidade. A função "client.publish(topic, payload)"é responsável por publicar a mensagem para o broker. Para os demais sensores, o algoritmo se repete com a alteração dos tópicos: para o segundo sensor, seriam "esp8266/sensor2/temperatura"e "esp8266/sensor2/umidade", e assim sucessivamente.

Depois do MQTT ser configurado, os dados são então publicados no tópico escolhido.

```
void publishMQTT(const char* topic, const char* payload) {
  if (client.publish(topic, payload)) {
    Serial.print("Mensagem enviada ao tópico ");
    Serial.println(topic);
  } else {
    Serial.print("Falha ao enviar mensagem ao tópico ");
    Serial.println(topic);
  }
}
```

Figura 12 – Código de publicação dos dados no MQTT

Esta função publica uma mensagem (payload) em um tópico específico no broker MQTT. Se a publicação der certo, uma mensagem de sucesso é impressa no console. Caso contrário, uma mensagem de erro é exibida.

Essa parte do código garante que os dados do sensor sejam publicados em tópicos específicos no broker MQTT, permitindo que outros dispositivos ou serviços possam subscrever e receber esses dados. A reconexão automática e o envio periódico de dados ajudam a manter a comunicação confiável e contínua.

3.2.3 Firebase

A parte do código que lida com o Firebase está configurada para enviar os dados lidos do sensor DHT11 (temperatura e umidade) para o banco de dados Firebase em tempo real.

#define FIREBASE_HOST "temperature-9ab56-default-rtdb.firebaseio.com"

#define FIREBASE_AUTH "AIzaSyBu_fHooG-QFKhNhpqUCIxQVCDBGaqabSs"

Essas linhas definem as constantes FIREBASE_HOST e FIREBASE_AUTH, que são, respectivamente, a URL do banco de dados Firebase e a chave de autenticação necessária para se conectar a ele.

Depois de configurado o Firebase, os dados são enviados d/entro da função loop(), apos a leitura dos dados do sensor:

```
if (Firebase.setFloat(firebaseData, "/sensor1/temperatura", temperatura)) {
  Serial.println("Temperatura enviada com sucesso");
} else {
  Serial.println("Falha ao enviar temperatura");
  Serial.println(firebaseData.errorReason());
}
```

Figura 13 – Código de envio dos dados ao Firebase

Firebase.setFloat(firebaseData, "/sensor1/temperatura", temperatura): Envia o valor da temperatura para o caminho especificado no banco de dados Firebase. Se a operação der certo, imprime "Temperatura enviada com sucesso", caso contrário, imprime uma mensagem de erro.

Firebase.setFloat(firebaseData, "/sensor1/umidade", umidade): Envia o valor da umidade para o caminho especificado no banco de dados Firebase. Se a operação for der cert, imprime "Umidade enviada com sucesso", caso contrário, imprime uma mensagem de erro.

Firebase.setString(firebaseData, "/sensor1/hora", formattedTime): Envia a hora formatada para o caminho especificado no banco de dados Firebase. Se a operação der certo, imprime "Hora enviada com sucesso", caso contrário, imprime uma mensagem de erro.

A parte do código referente ao Firebase é responsável por garantir que os dados coletados pelos sensores sejam enviados em tempo real, onde podem ser armazenados e posteriormente analisados. A integração com o Firebase facilita a centralização e a visualização dos dados, permitindo que eles sejam acessados de qualquer lugar com uma conexão a internet. Além disso, a reconexão automatica ao Wi-Fi ajuda a manter a conexão estavel e a garantir a continuidade do envio dos dados, mesmo em caso de interrupções na conexão.

3.2.4 Unity

A Unity desempenha um papel importante neste projeto, fornecendo a plataforma necesaria para a criação e visualização do ambiente virtual 3D. Como uma das ferramentas de desenvolvimento mais avançadas, a Unity oferece uma ampla variedade de funcionalidades que permite a construção de ambientes interativos e realistas. Com seu conjunto de bibliotecas e suporte a multiplas linguagens de programação, como C#, a Unity facilita a integração com

sistemas IoT e a modelagem de dados em tempo real. A ferramenta suporta o uso de realidade aumentada (RA), proporcionando uma visualização sobreposta ao ambiente físico, o que é essencial para monitorar e gerenciar variáveis ambientais de maneira rapisa. As possibilidades oferecidas pela Unity fazem dela uma escolha ideal para desenvolver soluções inovadoras, evidenciando sua flexibilidade como plataforma para este tipo de projeto.

O processo de desenvolvimento com Unity começa com a configuração do ambiente, que inclui a instalação do Unity Hub e do Unity Editor, além dos módulos necessários para RA. A escolha do SDK de RA dependerá das necessidades específicas do projeto, com o RA Foundation oferecendo suporte para funcionalidades básicas, enquanto o Vuforia pode ser usado para reconhecimento de imagens. O desenvolvimento inclui a criação de modelos 3D, a interação com o ambiente real e a implementação da lógica do aplicativo. A fase de testes é crucial, e a aplicação deve ser verificada em dispositivos físicos para garantir que funcione bem.

Para a configuração da realidade aumentada na unity, foi feito scripts para a configuração de cada dashboard, configuração da câmera e a leitura do sensor.

3.2.4.1 Dashboard

O script Dashboard cria um painel interativo que exibe dados em tempo real e gerencia a interface do usuário com base no rastreamento de um alvo utilizando Vuforia. Ele configura e atualiza textos com informações de horário, temperatura e umidade, e altera as cores dos botões conforme necessário. A detecção do alvo ativa ou desativa o canvas. Para cada dashboard existe um script que liga os dados de cada sensor para cada dashboard específico.

O dashboard é criado utilizando a biblioteca Vuforia. O script é implementado na classe Dashboard e implementa a interface ITrackableEventHandler para gerenciar o rastreamento de alvos.

O método Update é chamado a cada frame. Se o alvo for encontrado (targetFound é verdadeiro), o canvas é ativado e os textos de horário, temperatura e umidade são atualizados com os valores atuais. Caso contrário o canvas é desativado.

A interface ITrackableEventHandler requer a implementação do método OnTrackableStateChanged, que é chamado quando o estado de rastreamento do alvo muda. Se o novo estado for DETECTED, TRACKED ou EXTENDED_TRACKED, a variável targetFound é definida como verdadeira. Caso contrário ela é definida como falsa.

3.2.4.2 Configuração da Câmera

O script desenvolvido em C# para a plataforma Unity, utilizando a biblioteca Vuforia, tem como principal objetivo configurar a câmera do dispositivo para focar automaticamente no alvo detectado. Este script é implementado na classe CameraFocusController, que herda de MonoBehaviour, a classe base para todos os scripts em Unity.

Quando o script é iniciado, ele obtém uma instância do VuforiaARController e caso essa instância seja válida registra um callback que será acionado após o Vuforia ser iniciado. Este callback é responsável por configurar o foco automático da câmera.

O método que configura o foco automático é chamado no início e quando a aplicação é retomada após ser pausada. Para garantir a funcionalidade correta, o script verifica se o Vuforia já foi iniciado e, se for o caso reconfigura o foco automático da câmera.

A configuração do foco automático é realizada definindo o modo de foco da câmera para contínuo automático. Caso a configuração der certo, uma mensagem de sucesso é registrada. Se o dispositivo não suportar o foco automático, uma mensagem de aviso é exibida no console.

O script CameraFocusController assegura que a câmera do dispositivo mantenha o foco automático nos alvos detectados pela Vuforia, proporcionando uma melhor experiência de usuário em aplicações de realidade aumentada.

3.2.4.3 Leitura do Sensor

O script em C# para Unity, intitulado SensorReader, tem como principal objetivo ler dados de sensores do microcontrolador ESP8266 via protocolo MQTT e atualizr esses dados em um painel de controle na aplicação Unity. O script é implementado na classe SensorReader e utiliza bibliotecas de rede e MQTT para comunicação.

A função client_MqttMsgPublishReceived é chamado sempre que uma mensagem é recebida no tópico inscrito.

Este método:

- Converte a mensagem recebida de bytes para uma string.
- Registra a mensagem recebida no console de depuração.
- Converte a mensagem JSON para um objeto SensorData e armazena os dados no objeto sensorData.

Coroutine para Atualização com Atraso

O método UpdateComponent atualiza os valores de temperatura e umidade no painel de controle (Dashboard1) com os dados recebidos do sensor.

O script SensorReader estabelece uma conexão com um broker MQTT e se inscreve em um tópico específico, recebe e processa mensagens contendo dados do sensor e atualiza a interface do usuário com esses dados em tempo real.Esta abordagem é eficiente e eficaz para integrar dados de sensores IoT em uma aplicação de realidade aumentada ou qualquer outra aplicação interativa desenvolvida na Unity.

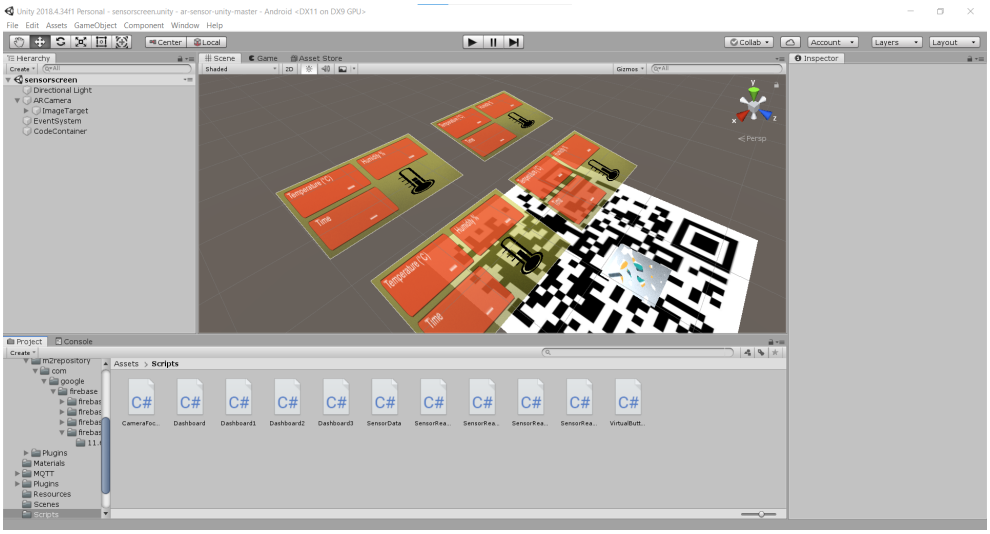


Figura 14 – Plataforma unity

4 Montagem do protótipo

4.0.1 Configuração ESP e sensor DHT11

Depois de tudo configurado foi montado o protótipo do projeto. O sensor de temperatura foi conectado aos pinos certos do NodeMCU ESP8266. Em seguida, o código foi carregado e testado no ESP para garantir que ele estava lendo os dados do sensor e enviando corretamente. Foi verificado se tudo estava funcionando conforme o planejado.

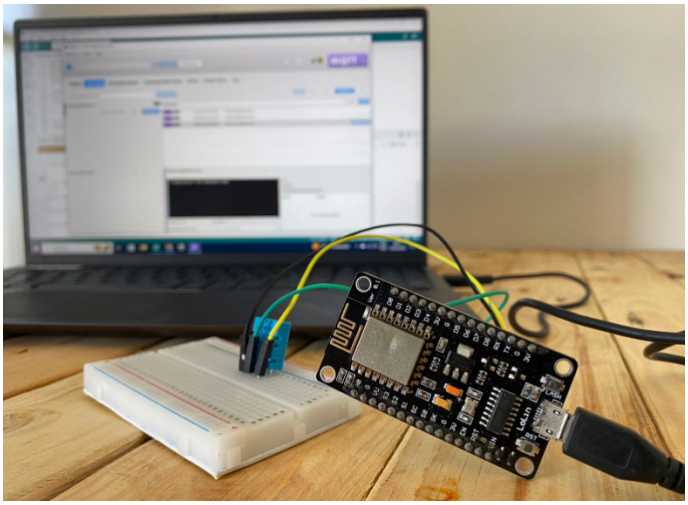


Figura 15 – Montagem do protótipo

Com o ESP e o sensor devidamente conectados, o proximo passo foi testar o envio dos dados para o MQTT e para o Firebase. Inicialmente, foi verificado se o ESP8266 conseguia se conectar à rede Wi-Fi e sincronizar com o servidor NTP para obter a hora correta. Em seguida, o sensor de temperatura começou a capturar as leituras do ambiente.

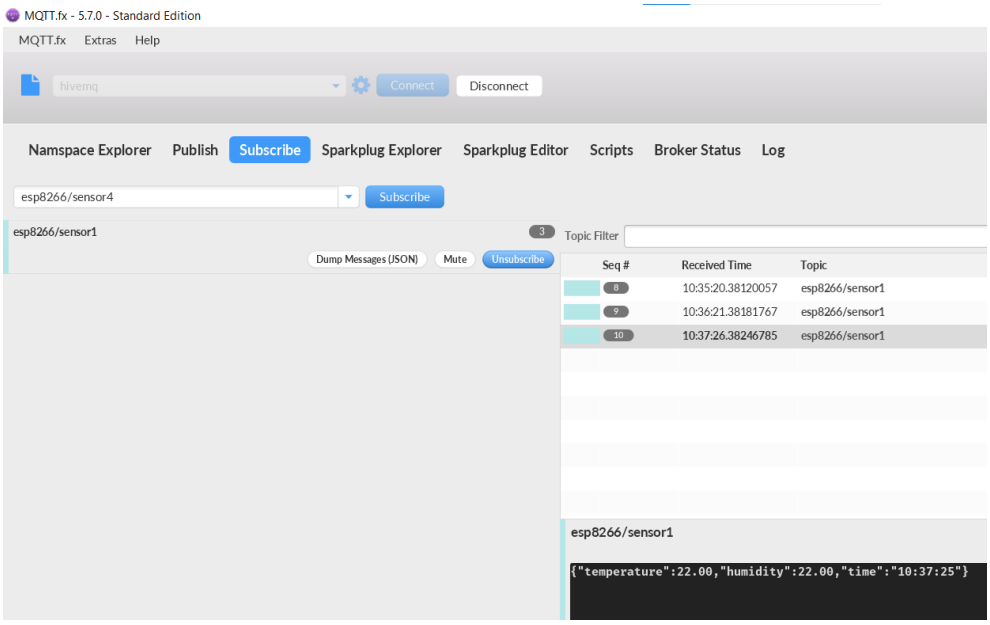


Figura 16 – Recebimento de dados MQTT

O código foi ajustado para garantir que os dados do sensor fossem enviados periodicamente para o broker MQTT, permitindo a comunicação eficiente entre o microcontrolador e outros dispositivos ou serviços conectados. Simultaneamente, os dados foram enviados para o Firebase, que atuou como uma base de dados em nuvem para armazenar as informações coletadas.

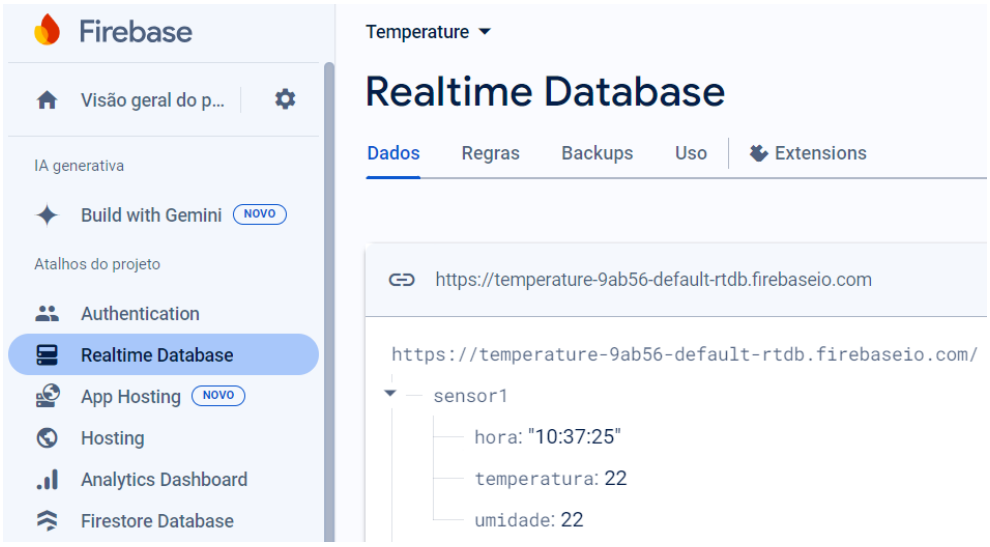


Figura 17 – Recebimento de dados Firebase

Durante os testes, observou-se se os dados eram transmitidos corretamente e se apareciam no Firebase como esperado. Também foi verificado o recebimento das mensagens no broker

MQTT para assegurar que a comunicação estava funcionando sem problemas. A estabilidade da conexão e a precisão dos dados foram monitoradas para garantir que o sistema estivesse operando de acordo com os requisitos estabelecidos. Todos esses testes foram fundamentais para confirmar o funcionamento adequado do sistema de monitoramento e garantir a integração eficiente entre os diferentes componentes do projeto.

4.0.2 Configuração Unity

A próxima etapa do projeto é fazer com que a Unity se inscreva no tópico do primeiro sensor de temperatura no broker MQTT para coletar os dados e exibi-los no dashboard correspondente. Primeiro, precisa-se desenvolver uma API na Unity que se conecte ao broker MQTT.

O processo começa com a configuração da conexão do script da Unity com o broker MQTT, ajustando as credenciais e parâmetros necessários. Depois, o script vai se inscrever no tópico específico onde os dados do sensor de temperatura são enviados. Isso permitirá que a Unity receba as mensagens em tempo real.

```
1 reference
21  private string topic = "esp8266/sensor1";
22
    0 references
23  void Start()
24  {
25      client = new MqttClient("broker.hivemq.com",1883 , false , null );
26
27
28      client.MqttMsgPublishReceived += client_MqttMsgPublishReceived;
29
30      string clientId = Guid.NewGuid().ToString();
31      client.Connect(clientId);
32
33
34      client.Subscribe(new string[] { topic }, new byte[] { MqttMsgBase.QOS_LEVEL_EXACTLY_ONCE });
35
36  }
37
```

Figura 18 – Unity se inscreve no tópico MQTT apropriado.

Uma vez que a inscrição no tópico esteja concluída, o proximo passo é coletar os dados do broker MQTT. O script deve processar essas mensagens e extrair as informações de temperatura.

Em seguida, integra-se esses dados ao dashboard virtual na Unity. Atualizando os elementos visuais no ambiente 3D para mostrar a temperatura atual. Com o dashboar configurado, as informações de temperatura, umidade e tempo são mostradas.



Figura 19 – Dashboard na realidade aumentada

Essa etapa é crucial para assegurar que os dados do sensor de temperatura sejam corretamente exibidos no ambiente virtual, proporcionando uma visualização precisa e dinâmica das informações de monitoramento.

4.0.3 Configuração completa

Após a configuração inicial e validação do funcionamento do primeiro ESP8266 e seu sensor de temperatura e umidade DHT11, o proximo passo envolve a configuração dos outros três dispositivos. Cada um desses dispositivos será configurado para operar de forma independente, utilizando tópicos distintos e caminhos específicos para armazenamento no Firebase, a fim de garantir que os dados de cada sensor sejam corretamente enviados e armazenados. Além disso foi feita a case na impressora 3D, como pode ser visto na figura **??** para otimizr o posicionamento dos protótipos.

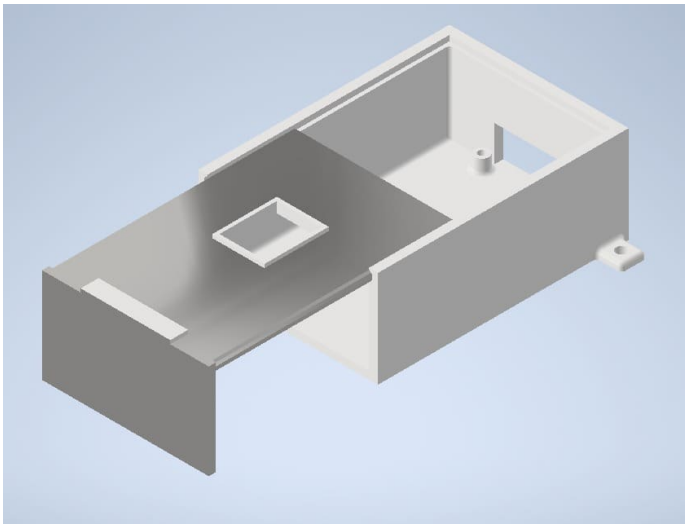


Figura 20 – Case feita em impressora 3D.

Cada ESP8266 será programado para se conectar à rede Wi-Fi, ler periodicamente a temperatura e a umidade do ambiente e enviar essas leituras para o Firebase, bem como para um broker MQTT. A programação incluirá a definição de tópicos exclusivos para cada dispositivo no broker MQTT, permitindo que os dados de cada sensor sejam enviados e recebidos sem interferência. No Firebase, os caminhos para armazenamento dos dados serão diferenciados para refletir as fontes dos dados e facilitar a organização e análise das informações coletadas.

Após a configuração dos quatro dispositivos, o sistema será testado de forma contínua para avaliar o seu desempenho ao longo do tempo. O objetivo é identificar possíveis falhas e otimizar o sistema para garantir que ele funcione de maneira mais confiável em diferentes condições ambientais.

Além disso, será monitorada a consistência das atualizações no Firebase e a resposta do sistema a eventuais desconexões ou problemas de comunicação. Ajustes e melhorias serão realizados conforme necessário para assegurar que o sistema de monitoramento ambiental opere conforme o esperado, fornecendo dados precisos e em tempo real.

5 Resultados

Nesta seção, os resultados obtidos ao longo do desenvolvimento deste trabalho serão apresentados e analisados, visando avaliar o cumprimento dos objetivos estabelecidos. Serão explorados os dados coletados e as metodologias aplicadas, bem como a eficácia das soluções implementadas. Também serão discutidas as implicações dos resultados tanto no contexto industrial quanto acadêmico, além de serem sugeridas direções para pesquisas futuras baseadas nas conclusões deste estudo.

5.1 Protótipos montados

Na Figura 21, estão representados todos os protótipos em funcionamento, demonstrando como cada dispositivo está operando e enviando dados. A figura ilustra claramente a integração dos sensores de temperatura e umidade com os microcontroladores ESP8266, e como esses dispositivos transmitem suas leituras em tempo real. Os dados coletados são enviados para o Firebase e o broker MQTT, e a visualização abrange tanto a interface do sistema de monitoramento quanto a representação dos dados em tempo real no ambiente virtual. Esta representação gráfica permite observar a interação entre os diferentes componentes do sistema, destacando o fluxo de dados e a comunicação entre os dispositivos e as plataformas de armazenamento e visualização.

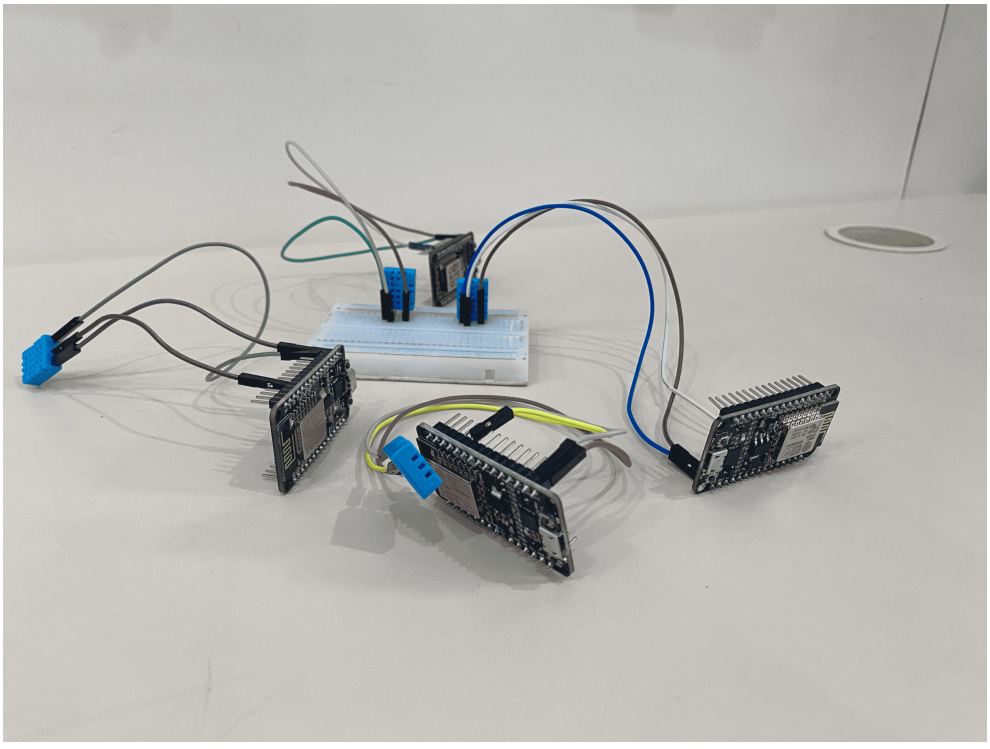


Figura 21 – Protótipos

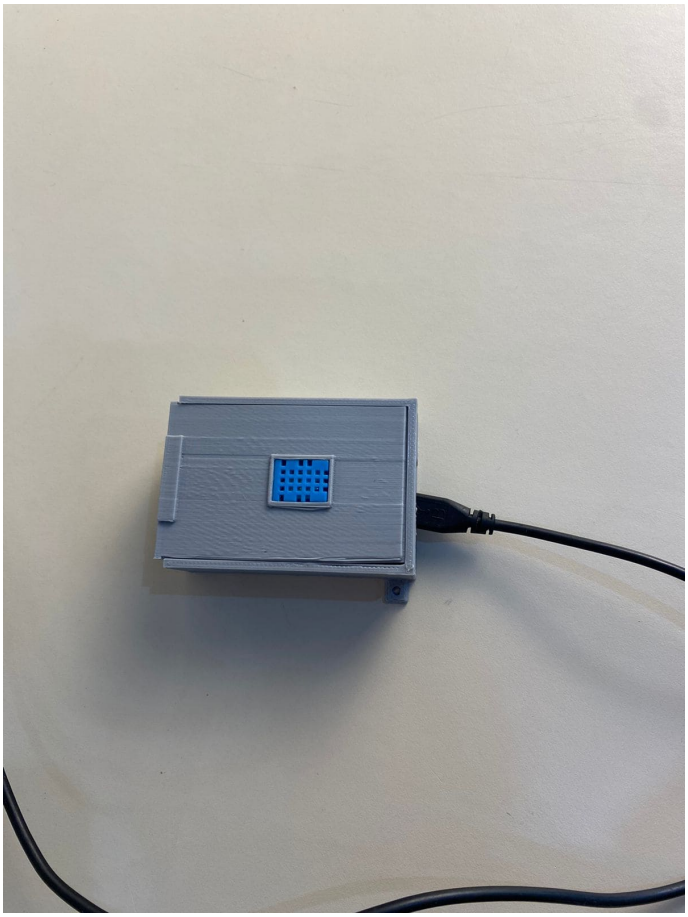


Figura 22 – Protótipo



Figura 23 – Protótipo

5.2 MQTT

Conforme ilustrado na figura 24, os dados coletados pelo sensor são transmitidos para o broker MQTT. O sistema foi configurado para enviar automaticamente as informações de temperatura e umidade para o MQTT sempre que o sensor realiza uma leitura. Essa transmissão continua é importante para garantir que os dados estejam sempre atualizados e acessíveis.

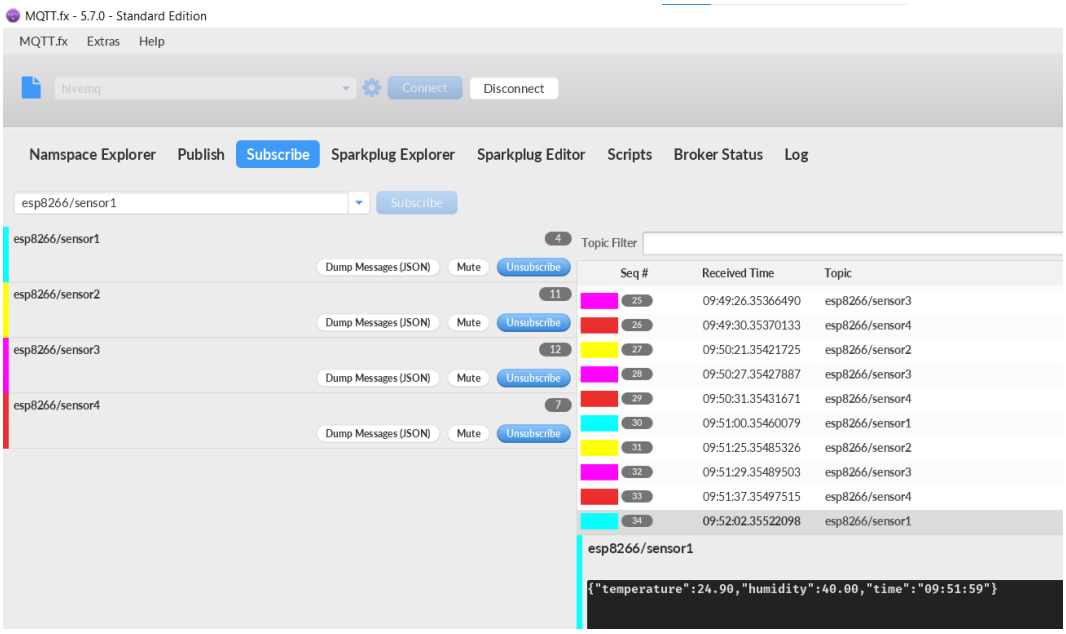


Figura 24 – Dados sendo recebidos pelo Firebase

Além disso, os dados podem ser visualizados em forma de gráficos, o que facilita a análise e interpretação das informações coletadas. Esses gráficos permitem identificar facilmente tendências e variações no ambiente ao longo do tempo, dando uma visão clara e das condições monitoradas. Além dos gráficos, os dados também podem ser acessados através da realidade aumentada, oferecendo uma abordagem interativa e intuitiva para a visualização das informações.

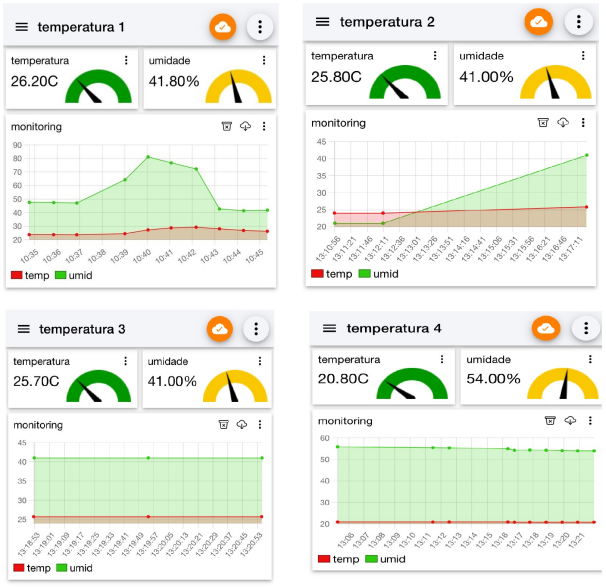


Figura 25 – Gráfico dos sensores

5.3 Firebase

Conforme pode ser observado na figura 28, os dados coletados pelo sensor são enviados diretamente para o Firebase. O sistema foi configurado para enviar automaticamente as informações de temperatura e umidade. Cada vez que o sensor realiza uma leitura, os dados são transmitidos imediatamente para o Firebase. Essa funcionalidade é essencial para permitir o monitoramento em tempo real das condições do ambiente. Além disso, o Firebase possibilita o armazenamento dos dados por longos períodos, o que é útil para a análise de tendências e padrões no futuro. A configuração também foi realizada de modo a organizar os dados de forma clara, facilitando sua visualização.



Figura 26 – Dados sendo recebidos pelo Firebase

5.4 Dashboard unity

Para mostra os dados em tempo real na Unity, foi criado 4 dashboards, sendo um para cada sensor. Esses dashboards foram projetados para exibir as informações específicas de cada sensor, como temperatura e umidade, bem como a hora atual permitindo uma visualização clara e organizada.

Através de um sistema de tracking utilizando a função TrackableBehaviour, os dashboards

são ativados e desativados conforme o rastreamento do target, que é uma imagem. Quando o target é detectado pelo sistema, o dashboard se torna visível, mostrando os dados coletados em tempo real.

Cada dashboard é responsável por apresentar os dados coletados de forma detalhada e interativa. Assim, é possível monitorar as condições específicas de cada sensor de maneira eficiente. O sistema garante que as informações sejam mostradas somente quando o target estiver sendo rastreado corretamente.

Com isso, o usuário tem uma visão em tempo real dos dados coletados, facilitando a análise e o acompanhamento das condições monitoradas. Esse método proporciona uma forma prática e eficiente de visualizar as informações de todos os sensores envolvidos no projeto.

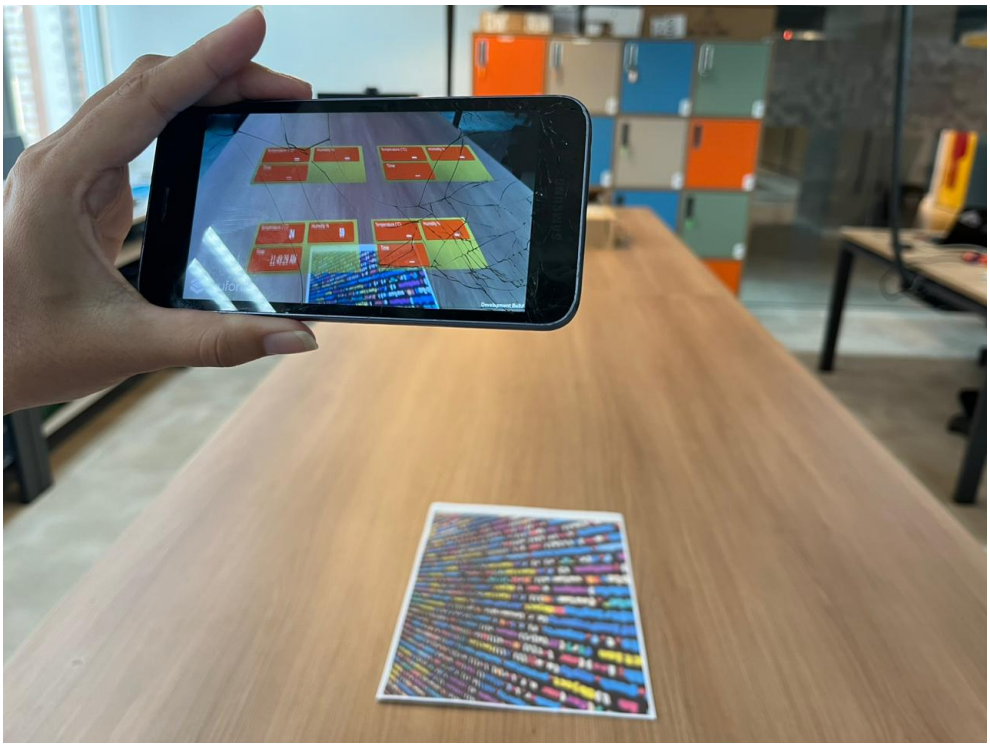


Figura 27 – Dashboard no celular



Figura 28 – Dashboard com todos os sensores enviando dados

6 Conclusão

Este trabalho de conclusão de curso abordou a criação de um sistema integrado de monitoramento que combina Realidade Aumentada (RA) e Internet das Coisas (IoT). O objetivo foi desenvolver uma solução inovadora utilizando o microcontrolador ESP8266, sensores de temperatura e umidade DHT11, e plataformas como Unity, Firebase e MQTT para criar uma visualização eficaz e interativa dos dados.

Durante o projeto, foi possível aplicar e integrar diversas tecnologias de forma prática. O ESP8266 desempenhou um papel fundamental como concentrador de dados, transmitindo informações para um ambiente virtual 3D desenvolvido no Unity. Essa integração proporcionou uma visualização 3D dos dados, permitindo uma análise detalhada e em tempo real das condições ambientais.

O projeto foi dividido em quatro etapas principais: captura dos dados dos sensores, análise e tratamento desses dados, transmissão via MQTT e Firebase, e visualização no ambiente virtual. Esse fluxo de trabalho garantiu que o sistema fosse eficiente e eficaz, oferecendo uma maneira inovadora de monitorar e interpretar dados ambientais.

Os resultados demonstraram como a integração de RA e IoT pode melhorar a gestão ambiental e a documentação técnica. A visualização em 3D não só torna os dados mais acessíveis e compreensíveis, mas também se mostra como uma ferramenta poderosa para a tomada de decisões em diferentes contextos industriais. Este trabalho aprofundou o conhecimento sobre RA e IoT e demonstrou como essas tecnologias podem ser integradas de maneira prática e inovadora.

6.1 Trabalhos futuros

Para aprimorar o sistema desenvolvido, considera-se as seguintes adições e melhorias:

- Verificação da Média da Temperatura da Sala;
- Alertas de Temperatura Fora da Faixa Programada;
- Integração com Outros Sensores Ambientais;
- Escalabilidade para Ambientes Maiores;
- Interface de Controle no Ambiente Virtual.

Referências

BRESSA, N. et al. What's the situation with situated visualization? a survey and perspectives on situatedness. *IEEE Transactions on Visualization and Computer Graphics*, v. 28, n. 1, p. 107–117, January 2022. Disponível em: <https://ieeexplore.ieee.org/document/9552238>. Citado na página 13.

CHANG, K.-M.; DZENG, R.-J.; WU, Y.-J. An automated iot visualization bim platform for decision support in facilities management. *Applied Sciences*, v. 8, n. 7, p. 1054, 2018. Disponível em: <https://www.mdpi.com/2076-3417/8/7/1086>. Citado na página 13.

DAS, M. *Tutorial: Augmented Reality with Internet of Things (IoT)*. 2021. Published May 5, 2021. Disponível em: <https://www.hackster.io/mithun-das/tutorial-augmented-reality-with-internet-of-things-iot-fb51e7>. Citado na página 26.

DAVE, B. et al. A framework for integrating bim and iot through open standards. *Automation in Construction*, v. 95, p. 35–45, 2018. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0926580517305964>. Citado na página 13.

EASTMAN, C. et al. *BIM Handbook: A Guide to Building Information Modeling for Owners, Designers, Engineers, Contractors, and Facility Managers*. 2nd. ed. [S.l.]: John Wiley & Sons Inc, 2011. 626 p. ISBN 9780470541371. Citado na página 16.

HUNKELER, U.; TRUONG, H. L.; STANFORD-CLARK, A. Mqtt-s – a publish/subscribe protocol for wireless sensor networks. *IBM Zurich Research Laboratory, Switzerland; IBM UK Laboratories, Hursley, UK*, IEEE, 2008. Disponível em: <https://ieeexplore.ieee.org/document/4554519>. Citado na página 20.

KIM, J. et al. Industrial augmented reality: Concepts and user interface designs for augmented reality maintenance worker support systems. In: *2020 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE, 2020. Disponível em: <https://ieeexplore.ieee.org/document/9288451>. Citado na página 18.

MIORANDI, D. et al. Internet of things: Vision, applications and research challenges. *Elsevier*, –, p. –, 2012. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1570870512000674>. Citado na página 16.

OLSHANNIKOVA, E. et al. Visualizing big data with augmented and virtual reality: challenges and research agenda. *Journal of Big Data*, v. 2, p. 1–22, 2015. Disponível em: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-015-0031-2>. Citado 2 vezes nas páginas 13 e 14.

SIMON, J. Augmented reality application development using unity and vuforia. *Interdisciplinary Description of Complex Systems*, v. 21, n. 1, 2023. Disponível em: <https://www.researchgate.net/publication/371462889>. Citado na página 21.

TEIZER, J. et al. Internet of things (iot) for integrating environmental and localization data in building information modeling (bim). In: *34th International Symposium on Automation and Robotics in Construction (ISARC 2017)*. [s.n.], 2017. Disponível em: <https://www.researchgate.net/publication/320166511>. Citado na página 13.

WU, X. A review of virtual reality technology. *Proceedings of the 2023 International Conference on Machine Learning and Automation*, 2023. Disponível em: <https://www.researchgate.net/publication/378435611_A_review_of_virtual_reality_technology>. Citado na página 19.

Anexos

ANEXO A – Código para Monitoramento de Temperatura e Umidade

```
#include <ESP8266WiFi.h>
#include <FirebaseESP8266.h>
#include <DHT.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <PubSubClient.h>

// Definições do WiFi
#define WIFI_SSID "wifi-zone-ufam-1"
#define WIFI_PASSWORD ""

// Definições do Firebase
#define FIREBASE_HOST "temperature-9ab56-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH "AIzaSyBu_fHooG-QFKhNhpqUCIxQVCDBGaqabSs"

// Definição do pino do sensor DHT e tipo de sensor
#define DHTPIN D4 // Pino onde o sensor está conectado no NodeMCU (GPIO2)
#define DHTTYPE DHT11 // Defina como DHT11 se estiver usando um DHT11

// Definições do MQTT
const char* mqtt_server = "broker.hivemq.com";
const char* mqtt_topic = "esp8266/sensor1";
const char* mqtt_topic_temp = "esp8266/sensor1/temperatura";
const char* mqtt_topic_hum = "esp8266/sensor1/umidade";

WiFiClient espClient2;
PubSubClient client(espClient2);

DHT dht(DHTPIN, DHTTYPE);
FirebaseData firebaseData;
FirebaseAuth auth;
FirebaseConfig config;

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", -14400, 60000); // Configuraçã
    o para o fuso horário de Manaus (UTC -4)

String mqtt_client_id;

void setup() {
```

```
40    Serial.begin(115200);
41    setup_wifi();
42
43    config.host = FIREBASE_HOST;
44    config.signer.tokens.legacy_token = FIREBASE_AUTH;
45
46    Firebase.begin(&config, &auth);
47    Firebase.reconnectWiFi(true);
48
49    dht.begin();
50
51    timeClient.begin(); // Inicializa o cliente NTP
52
53    mqtt_client_id = "esp8266_" + WiFi.macAddress();
54    client.setServer(mqtt_server, 1883);
55    connectToMqtt();
56  }
57
58  void setup_wifi() {
59    delay(100);
60    Serial.print("Conectando ao WiFi");
61    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
62
63    while (WiFi.status() != WL_CONNECTED) {
64      delay(500);
65      Serial.print(".");
66    }
67
68    Serial.println();
69    Serial.println("Conectado ao WiFi");
70    Serial.println("IP address: ");
71    Serial.println(WiFi.localIP());
72  }
73
74  void reconnect() {
75    while (!client.connected()) {
76      Serial.print("Attempting MQTT connection...");
77
78      if (client.connect(mqtt_client_id.c_str())) {
79        Serial.println("connected");
80        client.subscribe(mqtt_topic_temp);
81        client.subscribe(mqtt_topic_hum);
82      } else {
83        Serial.print("failed, rc=");
84        Serial.print(client.state());
85        Serial.println(" try again in 5 seconds");
86        delay(5000);
```

```
    }
  }
}

void connectToMqtt() {
  while (!client.connected()) {
    Serial.print("Conectando ao MQTT...");

    if (client.connect(mqtt_client_id.c_str())) {
      Serial.println("conectado");
    } else {
      Serial.print("falhou, rc=");
      Serial.print(client.state());
      Serial.println(" tentando novamente em 5 segundos");
      delay(5000);
    }
  }
}

void publishMQTT(const char* topic, const char* payload) {
  if (client.publish(topic, payload)) {
    Serial.print("Mensagem enviada ao tópico ");
    Serial.println(topic);
  } else {
    Serial.print("Falha ao enviar mensagem ao tópico ");
    Serial.println(topic);
  }
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  float temperatura = dht.readTemperature();
  float umidade = dht.readHumidity();

  if (isnan(temperatura) || isnan(umidade)) {
    Serial.println("Falha ao ler do sensor DHT!");
    return;
  }

  timeClient.update();
  String formattedTime = timeClient.getFormattedTime();

  Serial.print("Hora: ");
```

```
134   Serial.print(formattedTime);
135   Serial.print("  Temperatura: ");
136   Serial.print(temperatura);
137   Serial.print(" °C");
138   Serial.print("  Umidade: ");
139   Serial.print(umidade);
140   Serial.println(" %");
141
142   if (Firebase.setFloat(firebaseData, "/sensor1/temperatura", temperatura))
        {
143     Serial.println("Temperatura enviada com sucesso");
144   } else {
145     Serial.println("Falha ao enviar temperatura");
146     Serial.println(firebaseData.errorReason());
147   }
148
149   if (Firebase.setFloat(firebaseData, "/sensor3/umidade", umidade)) {
150     Serial.println("Umidade enviada com sucesso");
151   } else {
152     Serial.println("Falha ao enviar umidade");
153     Serial.println(firebaseData.errorReason());
154   }
155
156   if (Firebase.setString(firebaseData, "/sensor3/hora", formattedTime)) {
157     Serial.println("Hora enviada com sucesso");
158   } else {
159     Serial.println("Falha ao enviar hora");
160     Serial.println(firebaseData.errorReason());
161   }
162
163   //Envia a temperatura para o tópico MQTT específico
164   String tempMsg = String(temperatura);
165   char tempMessage[100];
166   tempMsg.toCharArray(tempMessage, 100);
167   publishMQTT(mqtt_topic_temp, tempMessage);
168
169   //Envia a umidade para o tópico MQTT específico
170   String humMsg = String(umidade);
171   char humMessage[100];
172   humMsg.toCharArray(humMessage, 100);
173   publishMQTT(mqtt_topic_hum, humMessage);
174
175   String msg = "{\"temperature\":" + String(temperatura) + ",\"humidity\":"
        + String(umidade) + ",\"time\":\"" + formattedTime + "\"}";
176   char message[100];
177   msg.toCharArray(message, 100);
178   publishMQTT(mqtt_topic, message);
```

```
179
180     delay(60000);
181 }
```

Código A.1 – Código para ESP8266 com Firebase e MQTT